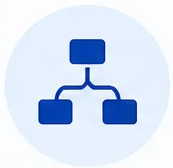


Uber Database Design & Implementation

A Simple Ride-Hailing System
Using **MySQL / MariaDB**



Well-Structured
Database



Normalized
Schema



Efficient &
Scalable



MySQL / MariaDB
Backend



Data Integrity
& Consistency



Users, Drivers,
Rides & More



Designed for
Clarity & Performance



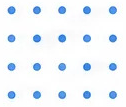


Table of Contents

Explore the sections of our database system design and implementation.

01



Introduction

02



System
Overview

03



Assignment
Requirements

04



Database
Schema

05



Table
Definitions

06



Constraints

07



Foreign Keys &
Relationships

08



How the
System Works

09



Example Ride
Conclusion



Well-Structured
Database Design



Data Integrity
& Consistency



Users, Drivers,
Rides & More

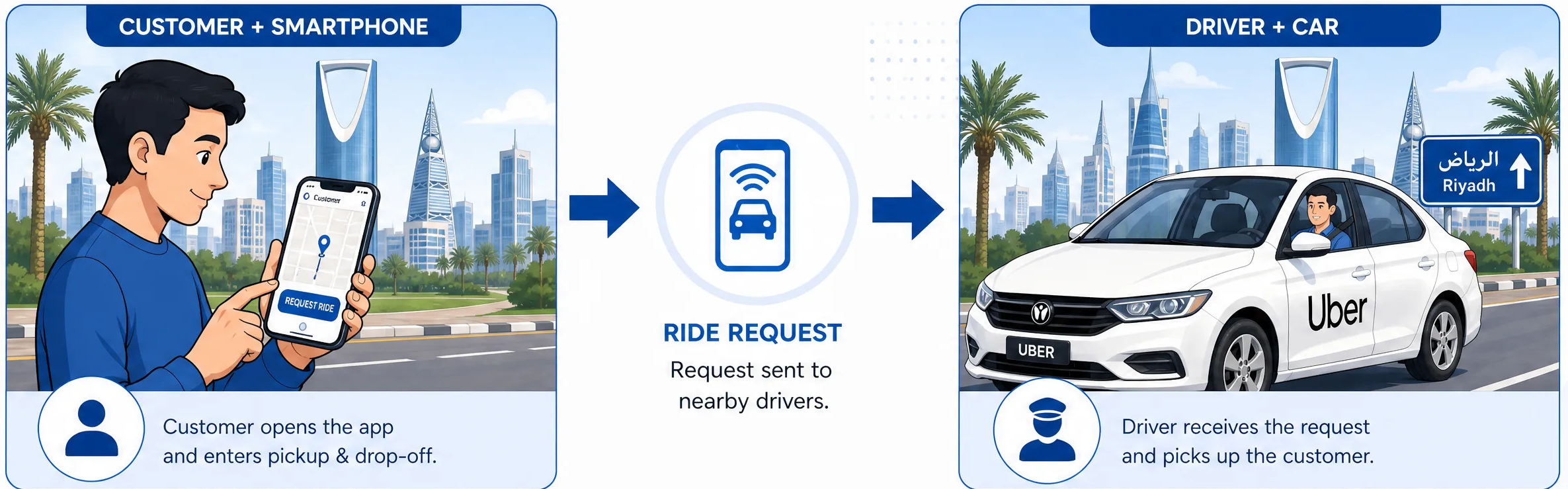


Designed for Clarity
& Performance



Uber-style Ride-Hailing

The customer requests a ride using the app, and the system connects them with a nearby driver.



Real-time matching. Smart routing. Reliable rides.

That's Uber-style simplicity.

System Workflow

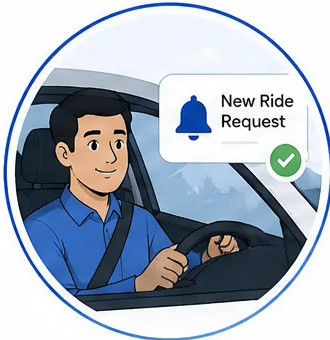
A seamless 6-step journey from request to completion.



1

Request ride

User enters pickup and drop-off location and requests a ride.



2

Driver accepts

Nearby driver receives the request and accepts the ride.



3

Trip starts

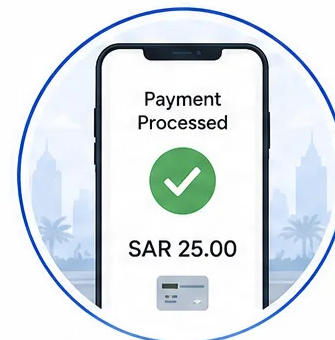
Driver arrives at pickup location and the trip begins.



4

Reach destination

Driver reaches the drop-off location safely.



5

Payment processed

Fare is calculated and payment is processed securely.



6

Trip completed

Trip ends, driver and rider can rate each other.



Designed for reliability, safety, and simplicity — delivering a smooth experience for both riders and drivers.



Rider First Experience



Secure Payments



Real-time Updates



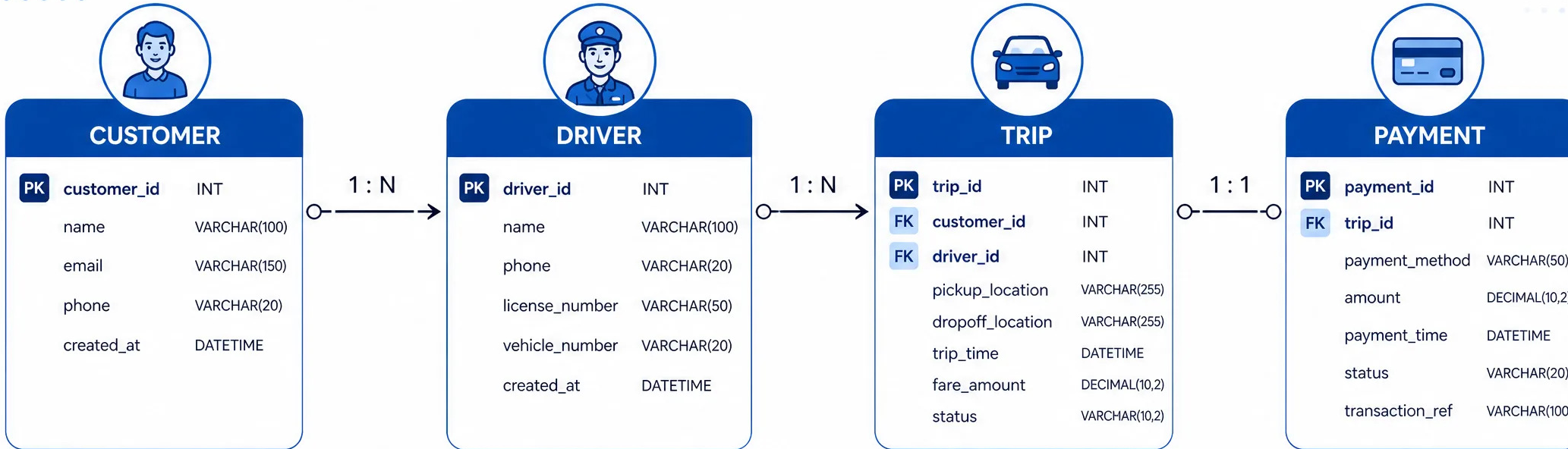
Optimized Performance

05 Database Schema / ER Diagram

Core entities and relationships powering the **Uber** ride-hailing platform.

PK Primary Key

FK Foreign Key



Customer ↔ Driver

1 : N

One customer can have many drivers over time.



Driver ↔ Trip

1 : N

One driver can complete many trips.



Customer ↔ Trip

1 : N

One customer can book many trips.



Trip ↔ Payment

1 : 1

Each trip has exactly one payment.



Note: PK = Primary Key, FK = Foreign Key. All relationships enforce referential integrity to ensure consistent and reliable data.



6 SQL Table Definitions

Core tables for the Uber ride-hailing system database.

CUSTOMER

```
1 CREATE TABLE customer (  
2   customer_id INT AUTO_INCREMENT PRIMARY KEY,  
3   full_name   VARCHAR(100) NOT NULL,  
4   email       VARCHAR(100) UNIQUE NOT NULL,  
5   phone       VARCHAR(20) UNIQUE NOT NULL,  
6   created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
7   status      ENUM('active','inactive') NOT NULL DEFAULT 'active'  
8 );
```

DRIVER

```
1 CREATE TABLE driver (  
2   driver_id INT AUTO_INCREMENT PRIMARY KEY,  
3   full_name  VARCHAR(100) NOT NULL,  
4   phone     VARCHAR(20) UNIQUE NOT NULL,  
5   email     VARCHAR(100) UNIQUE,  
6   license_number VARCHAR(50) UNIQUE NOT NULL,  
7   vehicle_plate VARCHAR(20) UNIQUE NOT NULL,  
8   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
9   ctatus    ENUM('active','inactive') NOT NULL DEFAULT 'active'  
10 );
```

TRIP

```
1 CREATE TABLE trip (  
2   trip_id INT AUTO_INCREMENT PRIMARY KEY,  
3   customer_id INT NOT NULL,  
4   driver_id INT NOT NULL,  
5   pickup_location VARCHAR(255) NOT NULL,  
6   dropoff_location VARCHAR(255) NOT NULL,  
7   pickup_time DATETIME NOT NULL,  
8   dropoff_time DATETIME NULL,  
9   fare_amount DECIMAL(10,2) NOT NULL,  
10  status      ENUM('requested','ongoing','completed','cancelled')  
11             NOT NULL DEFAULT 'requested',  
12  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13  FOREIGN KEY (customer_id) REFERENCES customer(customer_id),  
14  FOREIGN KEY (driver_id) REFERENCES driver(driver_id)  
15 );
```

PAYMENT

```
1 CREATE TABLE payment (  
2   payment_id INT AUTO_INCREMENT PRIMARY KEY,  
3   trip_id INT NOT NULL,  
4   amount DECIMAL(10,2) NOT NULL,  
5   payment_method ENUM('cash','card','wallet') NOT NULL,  
6   payment_status ENUM('pending','completed','failed','refunded')  
7                 NOT NULL DEFAULT 'pending',  
8   paid_at DATETIME NULL,  
9   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
10  FOREIGN KEY (trip_id) REFERENCES trip(trip_id)  
11 );
```



Well-Structured
Tables



Data Integrity
with Foreign Keys



Optimized for
Performance



Clear, Scalable
Schema Design



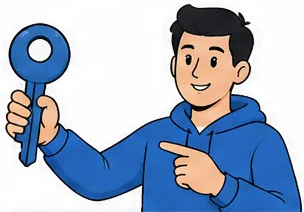
Constraints

Rules that enforce data integrity and relationships in our Uber database.



1 PRIMARY KEY

Uniquely identifies each record in a table.



RIDERS	
rider_id	name
1	Ahmed
2	Sarah
3	Faisal



Ensures every record is unique.

2 NOT NULL

Ensures a column cannot have NULL values.



DRIVERS	
driver_id	name
101	Omar
102	



Required field must always have a value.

3 UNIQUE

Ensures all values in a column are different.



USERS	
email	
ahmed@example.com	
sarah@example.com	
ahmed@example.com	



Prevents duplicate entries.

4 FOREIGN KEY

Links a column to the primary key in another table.



RIDES		RIDERS	
rider_id		rider_id	
1		1	
2		2	
3		3	



Maintains referential integrity across tables.



From Request to Payment. 6 Simple Steps.

A seamless ride-hailing experience across Riyadh.

1 Request

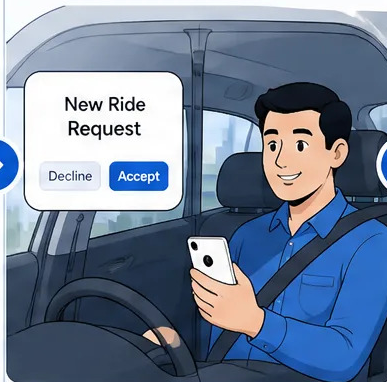
📍 Al Olaya, Riyadh



User opens the app and requests a ride.

2 Accept

📍 Nearby Driver



Nearby driver receives the request and accepts.

3 Pickup

📍 Al Olaya, Riyadh



Driver arrives at the pickup location. Rider gets in.

4 Trip

📍 En Route



Driver and rider travel to the destination.

5 Drop-off

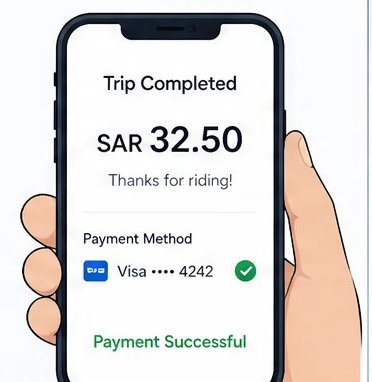
📍 Kingdom Centre, Riyadh



Rider reaches the destination. Trip ends.

6 Payment

📍 Paid Securely



Payment is processed securely in the app.



Safe. Reliable. Seamless.
Built for Riyadh. Designed for You.



Verified
Drivers



Real-time
Tracking



Secure
Payments



24/7
Support

Real-World Action → Database

Every action in the Uber system is stored in the right place.



REAL-WORLD ACTIONS



Customer request

A customer opens the app and requests a ride.



Driver info

A driver registers and keeps their profile updated.



Ride

A ride is created when a driver accepts the trip.



Payment

The customer pays for the completed ride.

DATABASE TABLES

CUSTOMER



Stores customer details like name, phone, email, and preferences.



DRIVER



Stores driver details like name, vehicle, license, and rating.



TRIP



Stores trip details like pickup, drop-off, time, distance, and fare.



PAYMENT



Stores payment details like amount, method, status, and transaction ID.



All data is connected, consistent, and easy to use.



Right data in the right place



Accurate & consistent records



Faster processing & better performance



Secure, reliable & scalable

Key Takeaways

We designed a solid Uber database foundation ready for real-world use.

- ✓  Real-world design
- ✓  4 core tables
- ✓  Applied constraints
- ✓  Defined keys
- ✓  Built relationships
- ✓  Clear data representation



Well-Structured
Database



Data Integrity
& Consistency



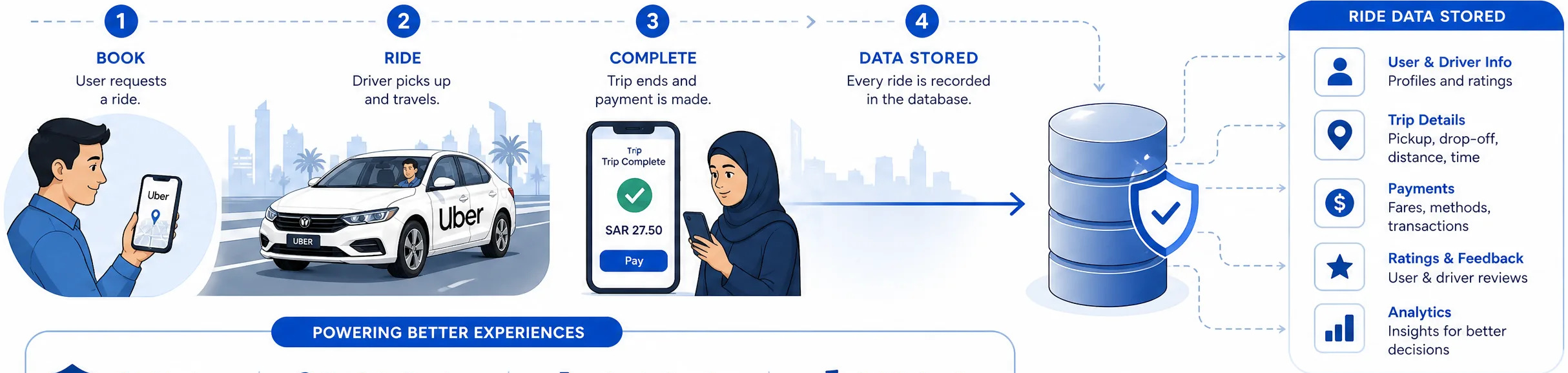
Users, Drivers,
Rides & More



Designed for
Clarity & Performance



A well-designed database turns every ride into structured, connected information.



POWERING BETTER EXPERIENCES



Data Integrity

Accurate, reliable and secure.



Better Experiences

Faster pickups, happier users.



Smarter Operations

Data-driven decisions improve efficiency.

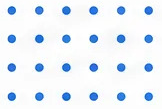


Scalable Growth

Built to support more rides, every day.



From every ride to real impact — data makes it possible.



12

Thank You

Thank You! Any Questions?



We appreciate your time
and interest in our project.
Let's build the future, together.



MySQL / MariaDB
Backend



Data Integrity
& Consistency



Users, Drivers,
Rides & More



Designed for
Clarity & Performance

